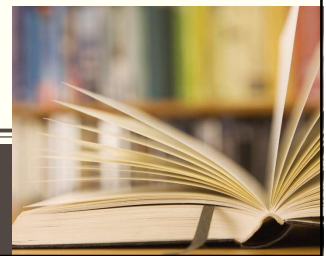
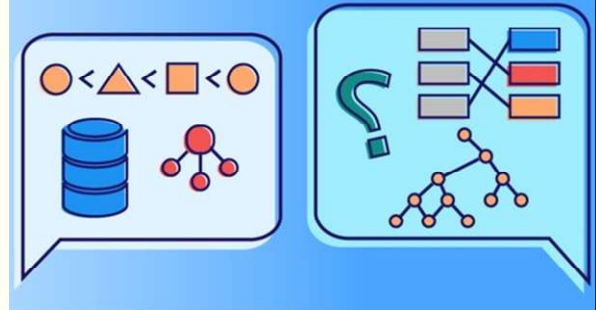


ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Agenda 14/09

- Analisis de Algoritmos.
- Características de los algoritmos.
- Complejidad.
- Analisis teórico de la complejidad.
- Principales ordenes de complejidad
- Punteros.
- Declaración e inicialización de punteros.
- Suma y resta de Punteros.
- Expresiones con punteros y aritmética de punteros.
- Punteros y arreglos.
- Notación de subíndices y punteros en arreglos.

2

Contraseña del estudiante

oovm6f

3

Analisis de Algoritmos - Introducción

Construcción de un
Algoritmo



- Podemos atacar el problema desde distintos puntos de vista,
- Aplicando distintas estrategias, y por tanto, llegando a soluciones algorítmicas distintas.

- Desde el punto de vista computacional, es necesario disponer de alguna forma de comparar una solución algorítmica con otra, para conocer cómo se comportarán cuando las implementemos, especialmente al trabajar con problemas "grandes".

La complejidad algorítmica es una métrica teórica que se aplica a los algoritmos en este sentido.

- Es un concepto fundamental para todos los programadores.
- Entender la complejidad es importante porque para resolver muchos problemas, utilizamos algoritmos ya diseñados. Conocer el valor de su complejidad puede ayudarnos a **escoger uno u otro**.

4

Características de los algoritmos

- Eficaces: Cumplen con el requerimiento solicitado
- Eficientes: Lo hacen mejor que otros
- Legibles, claros y bien estructurados
- Generales (capaces de resolver una clase de problemas lo más amplia posible), de uso y mantenimiento fácil.

EFICIENCIA: La eficiencia de un algoritmo es la propiedad mediante la cual un algoritmo debe alcanzar la solución al problema en el tiempo más corto posible y/o utilizando la cantidad más pequeña posible de recursos físicos.

1. Eficiencia temporal: el programa obtiene resultados rápido
2. Eficiencia espacial: hace buen uso de recursos.

5

Complejidad

- Al tiempo que consume un algoritmo para resolver un problema lo llamamos complejidad temporal.
- A la cantidad de memoria que utiliza el algoritmo la llamamos complejidad espacial.
- La complejidad espacial, en general, tiene mucho menos interés. El tiempo es un recurso mucho más valioso que el espacio.
- Así que cuando hablamos de complejidad, nos estamos refiriendo a complejidad temporal.

6

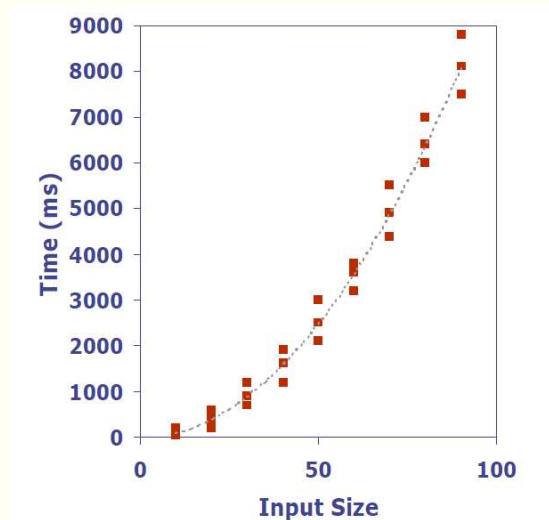
Complejidad

- ¿Qué medimos?: Tiempo de ejecución
- ¿De qué depende?:
 - Técnica utilizada para resolver el problema
 - Tamaño de la entrada
 - Otros factores (HW y SW)
- Características de la máquina (velocidad, tipo de procesador, RAM, SO...)
- Software usado en la implementación del programa (lenguaje, compilador, bibliotecas)
- ¿Cómo medirlo?:
 - Experimentalmente $\Rightarrow$ “a posteriori”
 - Estimándolo matemáticamente $\Rightarrow$ “a priori”.

7

Estudios Experimentales

- Escribir un programa implementando el algoritmo.
- Correr el programa con entradas de varios tamaños y composición.
- Usar un método tipo para tomar medidas precisas de cada corrida.
- Graficar los resultados.



8

Eficiencia de los algoritmos

Tiempo de ejecución:

- El tiempo de ejecución de un algoritmo típicamente crece con el tamaño de la entrada
- Pero no solo depende del tamaño, sino que influye el contenido de los datos
- Se distinguen 3 casos:
 - Mejor Caso: mínimo tiempo posible.
 - Peor Caso: máximo tiempo posible. Es el caso más representativo.
 - Caso Medio: tiempo promedio.

La mejor opción es analizar el peor caso.

9

Complejidad

- **Cálculo de $T(N)$:** El cálculo de la expresión $T(N)$ es fundamental para analizar la complejidad de los algoritmos, ya que nos ayuda a entender cómo se comporta el tiempo de ejecución en función del tamaño de la entrada N .
- **Cálculo estimado de $T(N)$:** se mide el tiempo de ejecución empíricamente. Requiere trabajo planificado y sistemático, presentación de los resultados y validez.
- **Cálculo de la expresión de $T(N)$:** se realiza un análisis matemático del algoritmo buscando obtener una expresión que describa el tiempo de ejecución. Se calcula una expresión exacta de $T(N)$ o una cota superior de la misma (expresión asintótica, orden de magnitud).

10

Complejidad

- **Expresión Exacta de $T(N)$** Se refiere a una fórmula precisa que describe el tiempo de ejecución del algoritmo para cualquier entrada de tamaño N . Ejemplo: si un algoritmo realiza 3 operaciones en cada una de las N iteraciones y tiene un costo adicional constante de 5, la expresión exacta podría ser: $T(N)=3N+5$. Esto nos da un valor específico para $T(N)$ dependiendo del tamaño N .
- **Cota Superior de $T(N)$** Se refiere a una estimación que proporciona un límite superior en el tiempo de ejecución en el peor de los casos, comúnmente expresada usando la notación Θ .
- **Ejemplo:** si un algoritmo tiene un comportamiento que, en el peor de los casos, no excede $5N^2+10N+3$, se puede afirmar que: $T(N)=\Theta(N^2)$
- Esto significa que aunque el tiempo exacto puede variar, en el peor de los casos, no crecerá más rápido que N^2 cuando N se vuelve muy grande.

11

Complejidad

- **Expresión Asintótica:** El análisis asintótico se centra en el comportamiento del algoritmo a medida que N tiende a infinito. Esto es útil porque permite comparar diferentes algoritmos independientemente de las constantes y factores menores que podrían afectar el tiempo de ejecución en entradas pequeñas.
- Cuando N es grande, términos de menor orden y constantes se vuelven menos significativos. Ejemplo: en la expresión $T(N)=3N^2+2N+1$, el término $3N^2$ dominará cuando N sea muy grande. Por lo tanto, podemos decir que: $T(N)=\Theta(N^2)$. Esto simplifica el análisis y permite enfocar en cómo crece el algoritmo a medida que el tamaño de entrada aumenta, lo que es esencial para tomar decisiones sobre la eficiencia.
- **Expresión Exacta:** Brinda el tiempo específico de ejecución en función de N .
- **Cota Superior:** Ofrece una estimación del peor caso, indicando cómo se comporta el algoritmo en situaciones extremas.
- **Análisis Asintótico:** Facilita la comparación de algoritmos al enfocarse en su comportamiento para entradas grandes, ignorando detalles menos relevantes en el contexto de crecimiento.

12

Analisis teórico de la Complejidad

- Como ejemplo se muestra cierto código para la búsqueda en un arreglo a con N elementos, de manera que se pueda determinar si un valor específico destino se encuentra en el arreglo:

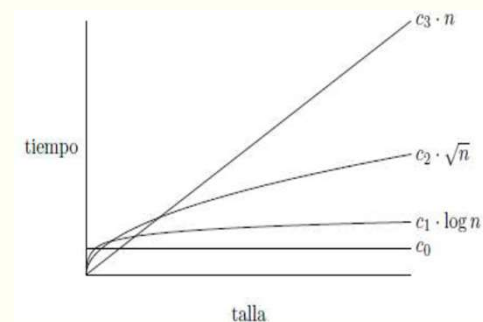
```
int i = 0;
bool encontrado = false;
while ((i < N) && !(encontrado))
    if (a[i] == destino)
        encontrado = true;
    else
        i++;
```

- $T(N) = 4 + 6N$ 2 inicializaciones, N repeticiones del ciclo en el peor caso (que destino no esté en el arreglo), 3 operaciones para evaluar cada condición del while y 3 operaciones dentro del cuerpo en el peor caso.

13

Gráfica de comparación de crecimiento de funciones Sublineales

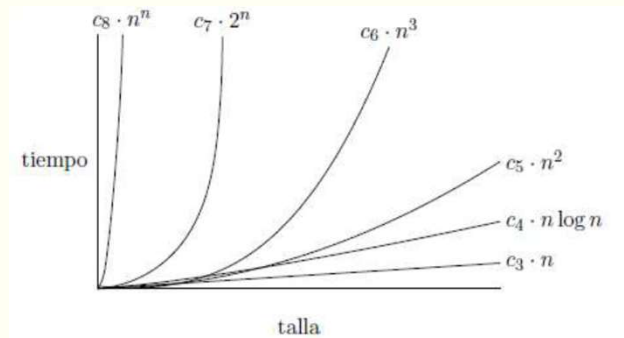
- Un algoritmo de coste constante ejecuta un n° constante de instrucciones. Un algoritmo que soluciona un problema en tiempo constante es lo ideal.
- El coste de un algoritmo logarítmico crece muy lentamente conforme crece n .
- Un algoritmo cuyo coste es $\sqrt{n}$ crece a un ritmo superior que otro que es logarítmico, pero no llega a presentar un crecimiento lineal.



14

Gráfica de comparación de crecimiento de funciones superlineales

- Un algoritmo cuyo coste es $(n \log n)$ presenta un crecimiento del coste ligeramente superior al de un algoritmo lineal.
- Un algoritmo de coste cuadrático empieza a dejar de ser útil para tallas medias o grandes.
- Un algoritmo de coste cúbico sólo es útil para problemas pequeños.
- Un algoritmo de coste exponencial raramente es útil.



15

Principales ordenes de Complejidad

Notación O	Orden
$O(1)$	Constante
$O(\log n)$	Logarítmico
$O(n)$	Lineal
$O(n^2)$	Cuadrático
$O(n^a)$	Polinomial ($a > 2$)
$O(a^n)$	Exponencial ($a > 1$)
$O(n!)$	Factorial

16

Complejidad

La complejidad es del algoritmo, no del código ni de los datos

No perder tiempo intentando optimizar el código, sino:
¡¡Tratar de optimizar el algoritmo!!

17

Cuál es la complejidad de ...

Ejercicio 1:

```
int suma = 0;
for (int i = 0; i < N; i++) {
    suma += arreglo[i];
}
```

$\Theta(N)$. El ciclo se repite exactamente N veces, por lo que el tiempo crece de forma lineal respecto al tamaño de la entrada.

18

Cuál es la complejidad de ...

Ejercicio 2:

```
int contador = 0;
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        contador++;
    }
}
```

$\Theta(N^2)$. Por cada iteración del ciclo externo (N veces), el ciclo interno se ejecuta N veces, resultando en un comportamiento cuadrático $N * N = N^2$.

19

Cuál es la complejidad de ...

Ejercicio 3:

```
// Matriz de N x N
for (int i = 0; i < N; i++) {
    cout << matriz[i][i];
}
```

$\Theta(N)$. Solo se accede a las posiciones donde la fila es igual a la columna ($\text{matriz}[i][i]$), por lo que el ciclo realiza únicamente N iteraciones.

20

Cuál es la complejidad de

- Obtener el menor de un arreglo de números “ordenado ascendentemente”.
- Buscar un número en un arreglo desordenado .
- Determinar el mayor de 3 números
- Buscar un elemento en una matriz
- Imprimir los valores de la diagonal principal de una matriz de N filas y N columnas
- Sumar los elementos de las columnas impares de una Matriz
- En una matriz cuadrada con sus diagonales principal y secundaria ordenadas ascendentemente, calcular el producto de los elementos máximos de cada diagonal
- Mezclar 2 arreglos

21

Cuál es la complejidad de

- Obtener el menor de un arreglo de números “ordenado ascendentemente”. $\Theta(1)$
- Buscar un número en un arreglo desordenado $\Theta(N)$
- Determinar el mayor de 3 números $\Theta(1)$
- Buscar un elemento en una matriz $\Theta(N^2)$
- Imprimir los valores de la diagonal principal de una matriz de N filas y N columnas $\Theta(N)$
- Sumar los elementos de las columnas impares de una Matriz $\Theta(N^2)$
- En una matriz cuadrada con sus diagonales principal y secundaria ordenadas ascendentemente, calcular el producto de los elementos máximos de cada diagonal $\Theta(1)$
- Mezclar 2 arreglos $\Theta(N)$

22

Para profundizar

Capítulo 14

Apunte de Complejidad (Campus)

Marzal – Gracia

Capítulo 6

Libro: Estructura de datos en C++

Joyanes

23

Algoritmos y Estructuras de Datos – AEDD – Asistencia 14/09

Contraseña del estudiante

oovm6f

24

Punteros – Un tipo de dato especial



25

Punteros

- Potentes, pero hay que dominar
- Simulan llamadas por referencia
- Estrecha relación con los arreglos y strings

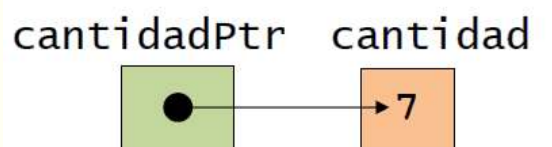
¿Para qué sirven?

- Permiten el manejo dinámico de memoria
- Permiten la implementación de estructuras de datos dinámicas
- Permiten implementar arrays

26

Punteros

- Contienen direcciones de memoria como sus valores.
- Las variables normales contienen un valor específico (referencia directa).
- Un puntero contiene la dirección de una variable que tiene un valor específico (referencia indirecta).



27

Definición e inicialización de variable Puntero

Definición de Punteros:

- Se usa * con variables puntero: `int *Punt1;`
- Define un puntero a un int (puntero de tipo int *)
- Varios punteros requieren el uso de un * antes de cada definición de variable:

```
int *Punt1, *Punt2;
```

- Se pueden definir punteros a cualquier tipo de datos.

Inicialización de Punteros:

- Los punteros se inicializan a 0, NULL, o a una dirección de memoria concreta.
- 0 ó NULL indican que apunta a nada (mejor NULL).

```
int *Punt1 = NULL;
```

```
int *Punt1 = 0;
```

28

Operadores de puntero

- *: operador de indirección/desreferencia
 - Retorna el contenido de la variable cuya dirección está almacenada en el puntero (operando de *), o
 - Retorna el contenido de la variable a la que apunta el puntero.
 - *yPtr retorna 5 (porque yPtr apunta a y)
 - * puede ser usado para asignaciones:
 - *yPtr = 7; /* cambia y al valor 7 */
 - El puntero desreferenciado (operando de *) debe ser una variable (no constante) a la izquierda del signo igual.

29

Expresiones con puntero

Sobre punteros se pueden realizar operaciones aritméticas:

- Incrementar/decrementar un puntero (++ , --)
- Sumar/restar un entero a un puntero (+ , += , - , -=)

Esto implica que la dirección almacenada en la variable tipo puntero se incrementa (o decrementa) en tantos bytes como corresponde al tipo de datos apuntado.

- Los punteros se pueden restar entre ellos

Estas operaciones tienen sentido cuando se apunta a un arreglo.

30

Suma de entero a puntero

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int v[5]={31,28,17,4,9};

    int *vPtr;
    vPtr = v;
    cout << *vPtr << endl;

    vPtr += 2;
    cout << *vPtr << endl;

    return 0;
}
```

```
31
17
```

31

Aritmetica de punteros en arreglos

Resta de punteros

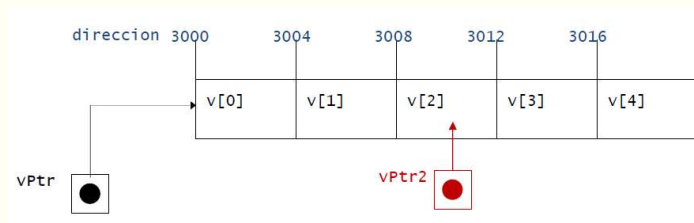
Retorna el número de elementos entre uno y otro.

▪ Si:

$vPtr = \&v[0];$

$vPtr2 = \&v[2];$

– $vPtr2 - vPtr$ retorna 2



32

Resta de puntero

```
#include <iostream>
using namespace std;

int main(int argc, char *argv[]) {
    int v[5]={31,28,17,4,9};

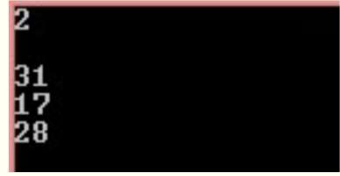
    int *vPtr;
    vPtr = v;

    int *vPtr2 = &v[2];

    cout << vPtr2 - vPtr << endl;

    cout << endl << *vPtr ;
    cout << endl << *vPtr2 ;
    vPtr2--;
    cout << endl << *vPtr2 ;

    return 0;
}
```



2
31
17
28

33

Expresiones con puntero y aritmética de punteros

Los punteros del mismo tipo se pueden asignar entre ellos. Ejemplo:

ptr = mi_arreglo;

La dirección del 1er elemento de mi_arreglo se asigna al puntero ptr, y de esta manera ambos son equivalentes.

- Si no son del mismo tipo, se debe usar un operador cast.
- Excepción: punteros sin tipo (genéricos)

void * Ptr; /* Declara un puntero genérico */

- Puntero genérico: no se asigna a ningún tipo de dato específico, sino que apunta a cualquier tipo de dato.
- No se necesita casting para convertir cualquier puntero a un puntero void.
- Un puntero void sí debe convertirse antes de asignarse a cualquier puntero (no puede asignarse directamente).
- Los punteros void no pueden ser desreferenciados.

34

Punteros y arreglos

Los arreglos y punteros se relacionan estrechamente

- Un nombre de arreglo es un puntero: contiene la dirección del primer elemento del arreglo.
- Un nombre de arreglo es un puntero constante: no se puede modificar, sólo se puede utilizar para acceder a los elementos del arreglo.
- Los punteros se pueden utilizar para acceder a un arreglo mediante subíndices.

Dado un arreglo `b[ 5 ]` y un puntero `bPtr`

- Se pueden asignar, mediante:
`bPtr = b;`
- El nombre del arreglo (`b`) es realmente la dirección del primer elemento del arreglo `b[ 5 ]`.
- `bPtr = &b[0]`; explícitamente asigna a `bPtr` la dirección del primer elemento de `b`.

35

Punteros y arreglos

Elemento `b[ 3 ]`

- Puede ser accedido mediante `*( bPtr + 3 )`
 - Donde 3 es el desplazamiento.
 - Notación puntero/desplazamiento
 - También puede ser accedido mediante `bptr[ 3 ]`
 - Notación puntero/subíndice
 - `bPtr[ 3 ]` es lo mismo que `b[ 3 ]`
- O, puede ser accedido mediante aritmética de punteros sobre el arreglo: `*( b + 3 )`.

36

Notación de subíndices y punteros con arreglos

```
#include <iostream>
using namespace std;

int main() {
    int b[5] = {1,3,5,7,9};
    int* p = b;
    int i;

    cout << "Salida con indices" << endl;
    for (i=0; i<5; i++)
        cout << b[i] << " ";

    cout << endl << endl;
    cout << "Salida con punteros" << endl;
    for (i=0; i<5; i++)
        cout << *(p+i) << " ";

    cout << endl << endl;
    cout << "Salida con punteros alternativa" << endl;
    for (i=0; i<5; i++)
        cout << p[i] << " ";

    cout << endl << endl;
    cout << "Salida con vector como puntero" << endl;
    for (i=0; i<5; i++)
        cout << *(b+i) << " ";

    return 0;
}
```

```
Salida con indices
1 3 5 7 9

Salida con punteros
1 3 5 7 9

Salida con punteros alternativa
1 3 5 7 9

Salida con vector como puntero
1 3 5 7 9
```

37

Otro ejemplo

```
#include <iostream>
using namespace std;

int main() {
    int t [2][3] = {{1,2,3},{4,5,6}};
    int* z = t[0];

    cout << z << endl;
    cout << &t[0][0] << endl;
    cout << "-----" << endl;

    cout << t[1][0] << endl;
    cout << *(z+3) << endl;
    cout << "-----" << endl;
    cout << &t[1][0] << endl;
    cout << z+3;

    return 0;
}
```

```
0x28ff04
0x28ff04
-----
4
4
-----
0x28ff10
0x28ff10
```

38

Algoritmos y Estructuras de Datos – AEDD – Asistencia 14/09

Contraseña del estudiante
oovm6f

39

Algoritmos y Estructuras de Datos – AEDD – Agenda 14/09

- ✓ Analisis de Algoritmos.
- ✓ Características de los algoritmos.
- ✓ Complejidad.
- ✓ Analisis teórico de la complejidad.
- ✓ Principales ordenes de complejidad
- ✓ Punteros.
- ✓ Declaración e inicialización de punteros.
- ✓ Suma y resta de Punteros.
- ✓ Expresiones con punteros y aritmética de punteros.
- ✓ Punteros y arreglos.
- ✓ Notación de subíndices y punteros en arreglos.

¡OBJETIVO CUMPLIDO!

40

Para profundizar

Capítulo 12

“Apuntadores”

Libro: “C++ para ingeniería y ciencias” 2da edición -

Gary J. Bronson, pág. 667

41



**ALGORITMOS Y
ESTRUCTURA DE
DATOS**

Comisión F – 2026 – Cursado Anual

¿DUDAS?

42

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¡Muy buena semana!

